

Presented by:

Achi Lara, Beșliu Beatrice, Antonescu Ema
(CTI-RO C1.1)

Proiect 1 la Calculatoare Numerice: **8-bit ALU Simulation**

Obiectivele proiectului

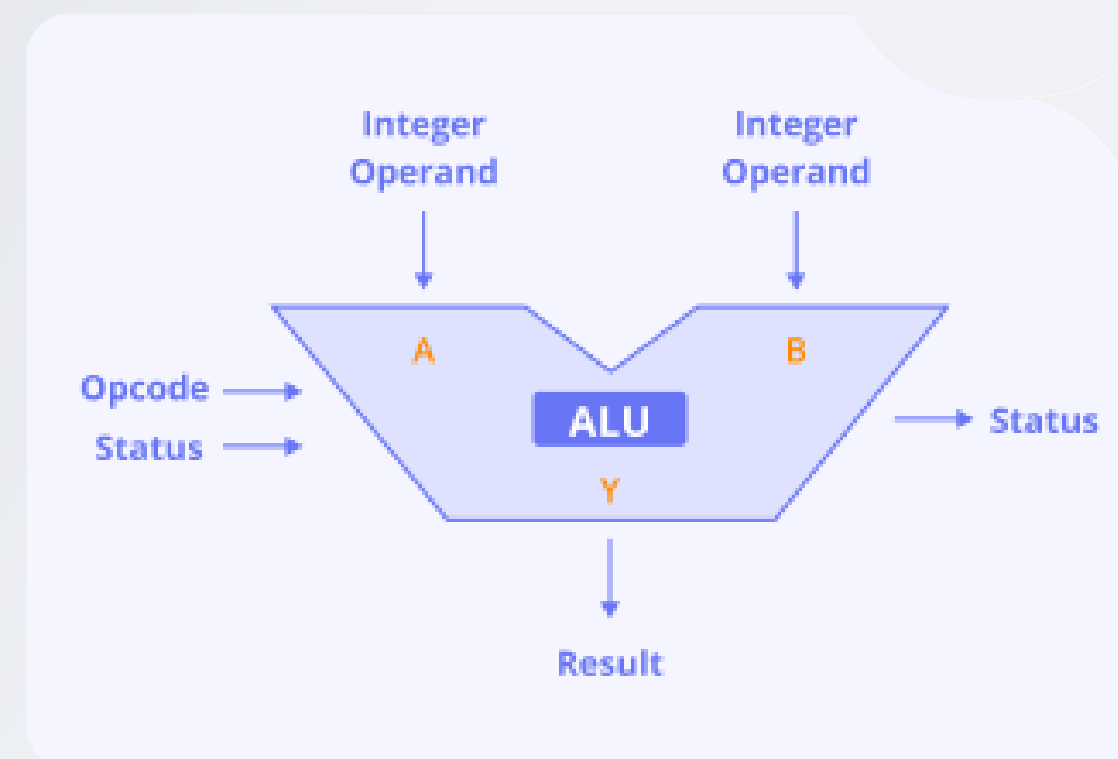
Acest proiect constă în proiectarea și simularea unei Unități Aritmetice și Logice (ALU) pe 8 biți, utilizând limbajul Verilog HDL. Scopul principal este realizarea unei arhitecturi structurale, în care fiecare operație aritmetică este implementată prin module hardware distincte, interconectate logic.

ALU-ul proiectat suportă patru operații fundamentale: adunare, scădere, înmulțire și împărțire. Selectarea operației se realizează prin intermediul unui semnal de control (opcode), care determină rezultatul final transmis la ieșire.

Obiectivul proiectului este de a evidenția modul de implementare hardware al operațiilor aritmetice, precum și integrarea acestora într-un sistem coerent, testabil și extensibil.

Ce este un ALU?

Un ALU (Arithmetic Logic Unit), sau Unitatea Aritmetică și Logică, este o componentă esențială a procesorului care se ocupă de executarea operațiilor aritmetice și logice asupra datelor binare. Aceasta primește date binare și un semnal de control care stabilește operația ce trebuie realizată. ALU poate efectua calcule precum adunarea, scăderea, înmulțirea și împărțirea, dar și operații logice precum AND, OR, XOR și NOT. Rezultatul obținut este trimis înapoi către procesor pentru a fi folosit în execuția instrucțiunilor, contribuind astfel atât la efectuarea calculelor, cât și la luarea deciziilor logice în sistemele digitale.



Decizie de proiectare - arhitectură modulară

Pentru realizarea ALU-ului, am ales o arhitectură modulară, în care fiecare operație este implementată într-un bloc separat. Această organizare împarte sistemul în componente distincte pentru adunare, scădere, înmulțire și împărțire, ceea ce face designul mai clar și mai ușor de urmărit. Fiecare modul poate fi dezvoltat și testat individual, fără a afecta celelalte părți ale sistemului. În plus, această abordare permite optimizarea separată a fiecărei operații, de exemplu utilizarea algoritmului Booth pentru înmulțire și SRT pentru împărțire. Rezultatul final este selectat printr-un multiplexor, în funcție de semnalul de control (opcode), ceea ce asigură flexibilitate în funcționarea ALU-ului.

Adunare

Operația de adunare este implementată în cadrul ALU-ului printr-un sumator de tip adder pe 8 biți, realizat modular cu ajutorul unei structuri de tip Ripple Carry Adder. Această arhitectură este construită pornind de la un full adder elementar, instanțiat iterativ pentru fiecare bit al operanzilor. Fiecare etapă calculează suma parțială și semnalul de transport (carry), care este propagat secvențial de la bitul cel mai puțin semnificativ către bitul cel mai semnificativ. Modulul este parametric pe 8 biți, ceea ce permite extinderea ușoară la alte dimensiuni prin modificarea lățimii de date. La final, rezultatul reprezintă suma corectă a celor doi operanzi, iar semnalul de overflow este determinat prin analiza ultimelor două valori de carry, indicând depășirea domeniului de reprezentare semnat.

Scădere

Operația de scădere este implementată utilizând metoda complementului față de 2, transformând astfel scăderea într-o operație de adunare. În implementarea hardware, acest lucru este realizat prin inversarea bit cu bit a celui de-al doilea operand ($\sim B$) și adăugarea constantei 1 prin semnalul de carry inițial ($C_{in} = 1$). Astfel, expresia $A - B$ devine $A + (\sim B + 1)$. Această abordare permite reutilizarea completă a infrastructurii de adunare, fără a necesita un circuit separat pentru scădere. Rezultatul este furnizat pe 8 biți, iar overflow-ul este calculat similar cu cazul adunării, prin analiza transportului final.

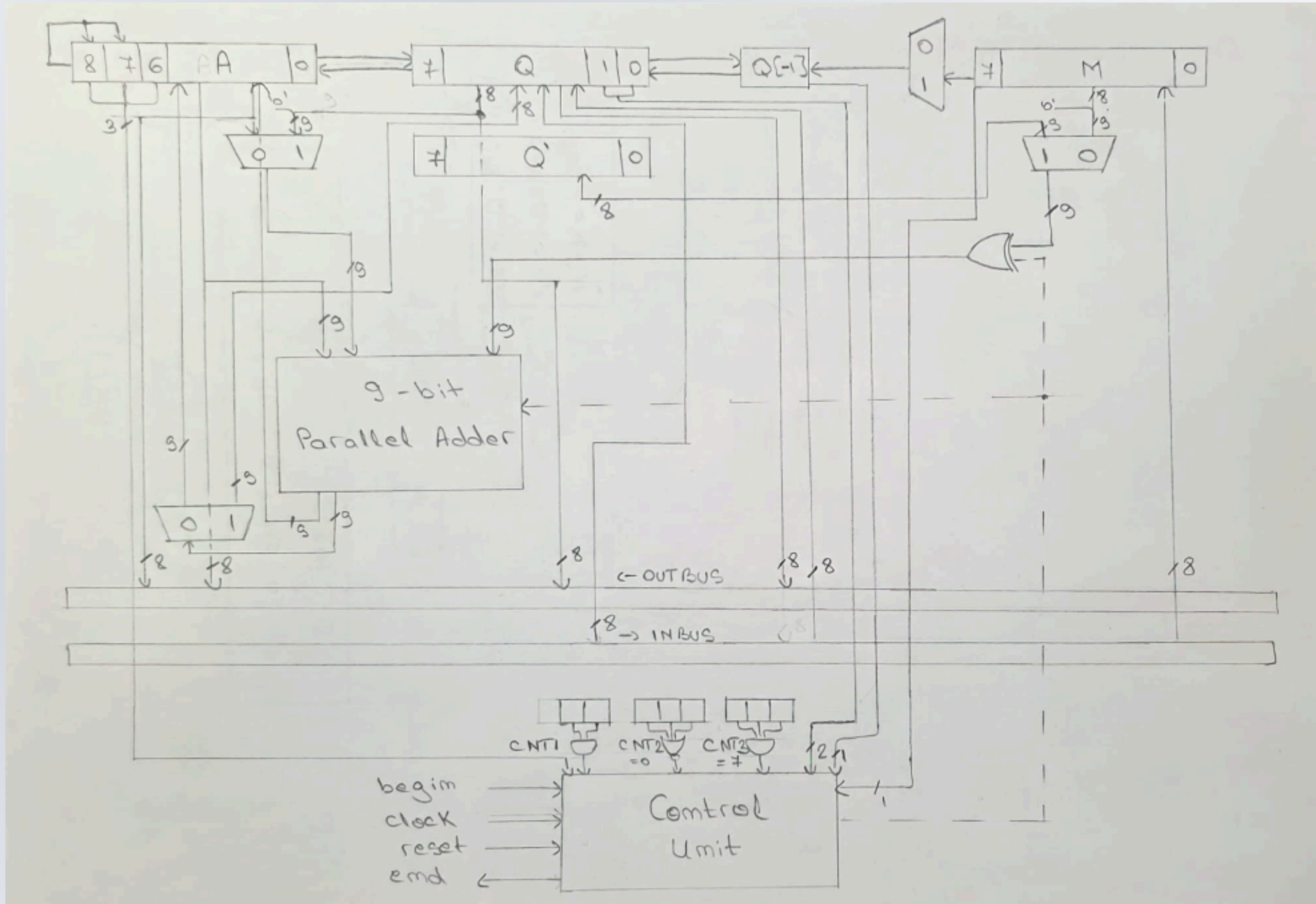
Înmulțire

Pentru implementarea înmulțirii a fost utilizat algoritmul Booth radix-4, ales pentru eficiența sa în reducerea numărului de operații necesare în comparație cu metoda clasică de înmulțire. Algoritmul procesează câte doi biți ai multiplicatorului simultan, prin gruparea acestora împreună cu un bit adițional de extensie. În funcție de combinația acestor biți, se generează termeni parțiali corespunzători operațiilor $+M$, $-M$, $+2M$ sau $-2M$. Fiecare produs parțial este apoi deplasat corespunzător (shift stânga) în funcție de poziția grupului de biți. La final, toate produsele parțiale sunt însumate pentru a obține rezultatul final pe 16 biți, asigurând reprezentarea corectă a valorilor semnate.

Împărțire

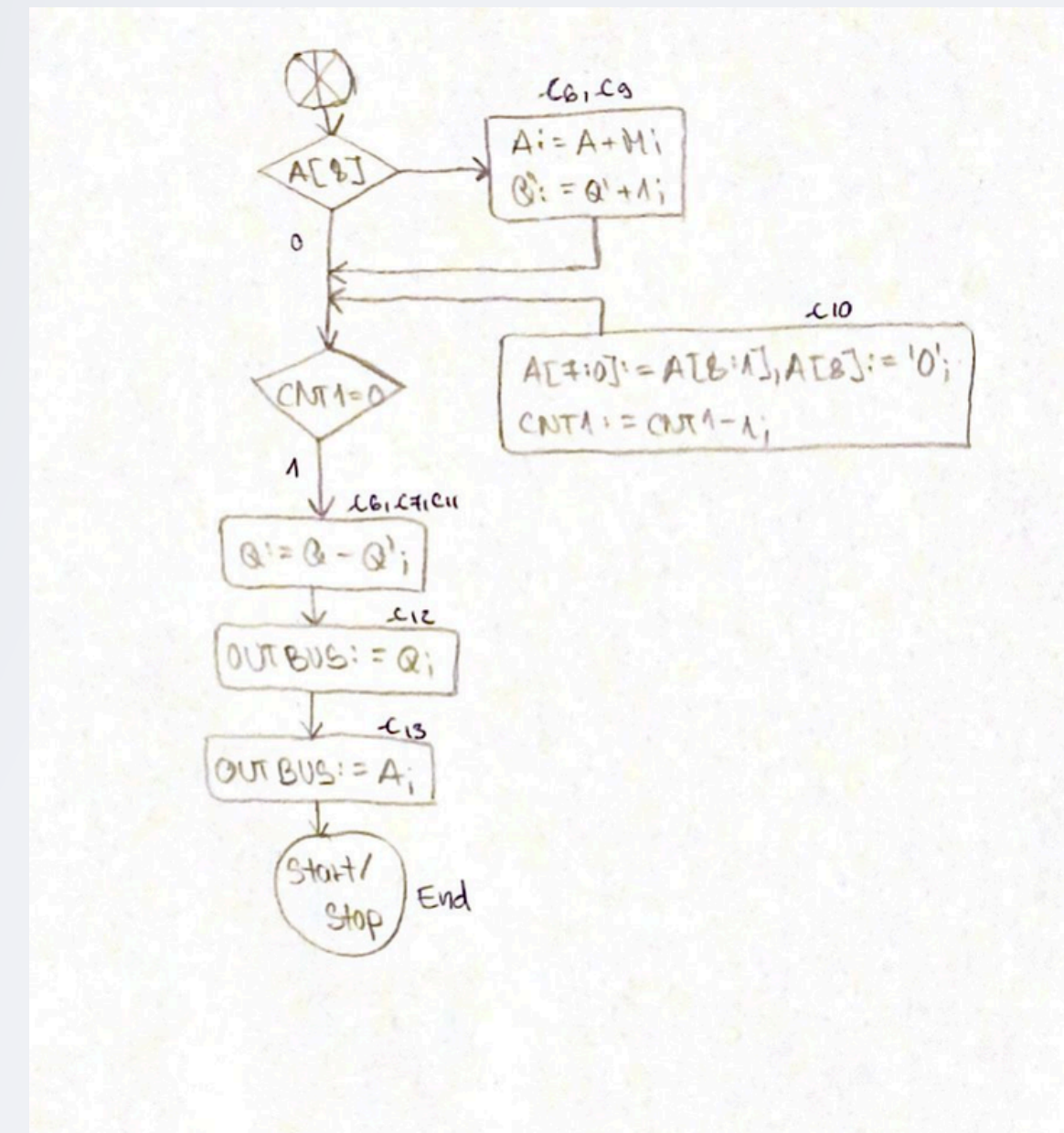
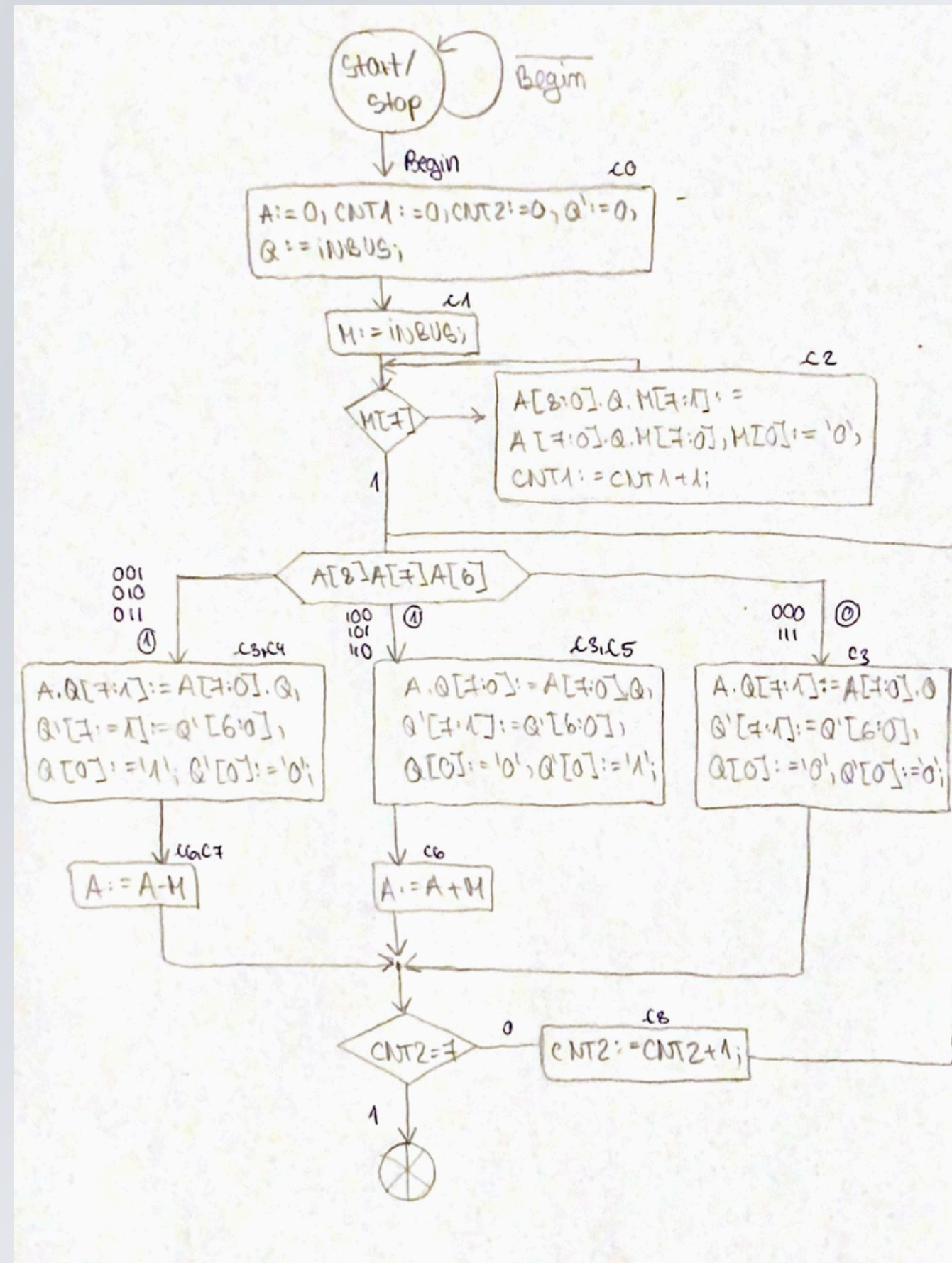
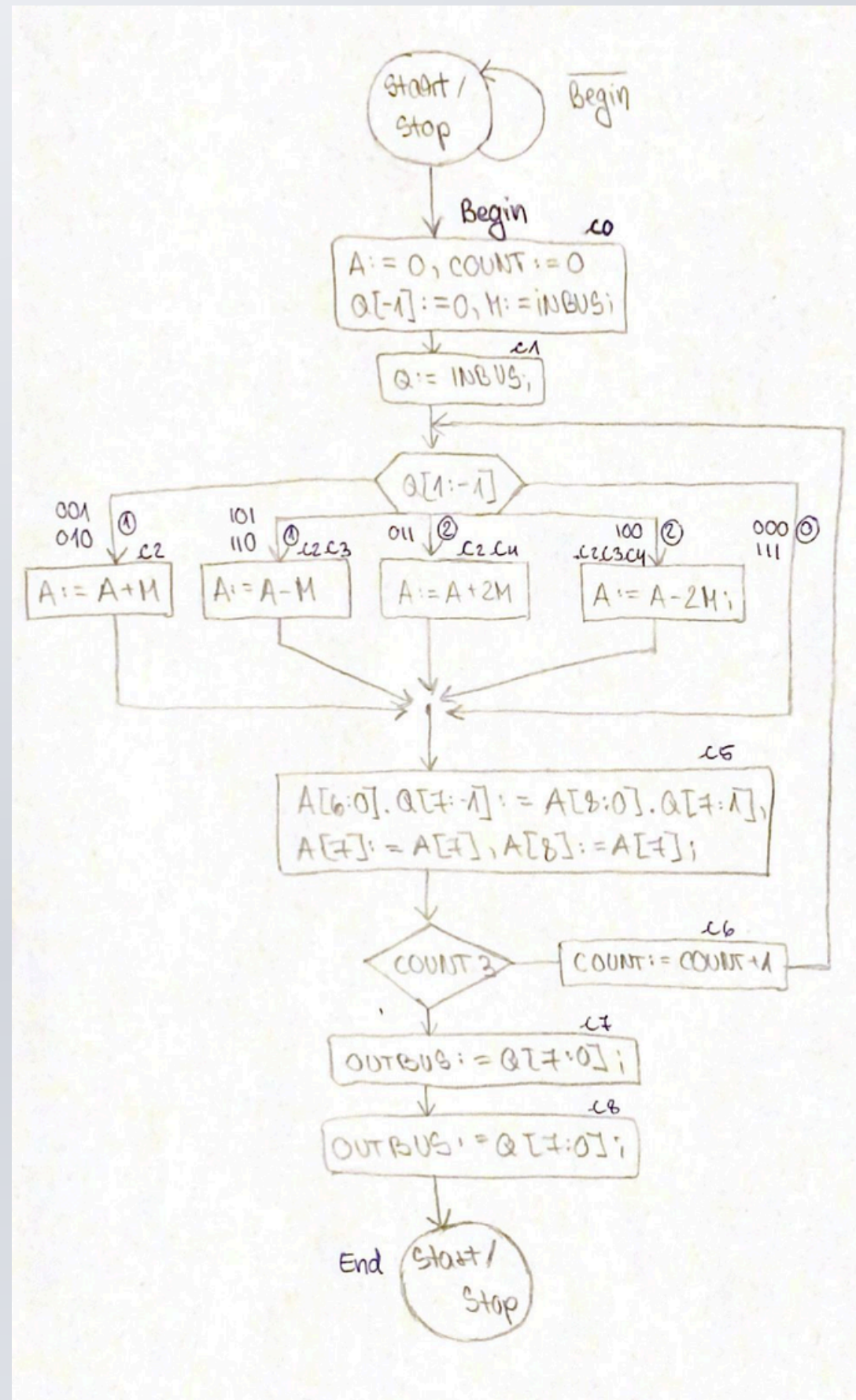
Pentru implementarea operației de împărțire a fost utilizat algoritmul non-restoring signed division, care permite calculul iterativ al câtului și restului printr-un proces secvențial pe 8 pași (corespunzători lățimii de 8 biți). La fiecare iterație, registrul combinat format din rest parțial (A) și cât (Q) este deplasat la stânga, iar în funcție de semnul registrului A se decide operația de adunare sau scădere a divizorului. Dacă restul parțial este pozitiv, se efectuează scăderea, iar dacă este negativ, se efectuează adunarea, asigurând astfel convergența algoritmului. Bitul de cât este actualizat la fiecare pas în funcție de semnul rezultatului intermediar. La finalul celor 8 iterații, se realizează o corecție a restului, dacă este necesar, iar semnele finale ale câtului și restului sunt ajustate în funcție de semnele operanzilor inițiali. Rezultatul final este furnizat sub forma câtului și restului pe 8 biți, împreună cu un semnal de detecție a împărțirii la zero.

Diagrama Hardware



Booth Radix-4

SRT Radix-2



Modul ALU

```
module alu_top (
    input clk, rst,
    input [1:0] opcode,
    input signed [7:0] A_in, B_in,
    output reg signed [7:0] result,
    output reg signed [15:0] result_mul,
    output reg signed [7:0] remainder,
    output reg overflow,
    output reg div_zero
);

    wire [7:0] sum, sub;
    wire ov_add, ov_sub;
    wire signed [15:0] mul;
    wire signed [7:0] quot, rem;
    wire dz;

    adder_8bit ADD_INST (A_in, B_in, 1'b0, sum, ov_add);
    adder_8bit SUB_INST (A_in, ~B_in, 1'b1, sub, ov_sub);
    booth_multiplier_radix4 MUL_INST (A_in, B_in, mul);
    divider_nonrestoring_signed DIV_INST (A_in, B_in, quot, rem, dz);

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            result      <= 0;
            result_mul <= 0;
            remainder  <= 0;
        end
    end
```

```
        overflow      <= 0;
        div_zero      <= 0;
    end else begin
        overflow <= 0;
        div_zero <= 0;

        case (opcode)
            2'b00: begin
                result      <= sum;
                overflow <= ov_add;
            end
            2'b01: begin
                result      <= sub;
                overflow <= ov_sub;
            end
            2'b10: begin
                result_mul <= mul;
                result      <= mul[7:0];
            end
            2'b11: begin
                result      <= quot;
                //remainder <= rem;
                div_zero    <= dz;
            end
        endcase
    end
end
endmodule
```

TESTBENCH

```
module tb;

    reg [1:0] op;
    reg signed [7:0] A, B;

    wire signed [7:0] result;
    wire signed [15:0] mul;
    wire ov, dz;

    reg clk = 0;
    always #5 clk = ~clk;

    reg rst = 0;

    alu_top uut(clk, rst, op, A, B, result, mul, , ov, dz);

    initial begin
        rst = 1; #10; rst = 0;

        // ADD
        op=2'b00; A=20; B=6; #20;

        // SUB
        op=2'b01; A=50; B=4; #20;
```

```
        // SUB
        op=2'b01; A=50; B=4; #20;

        // MUL
        op=2'b10; A=10; B=8; #20;

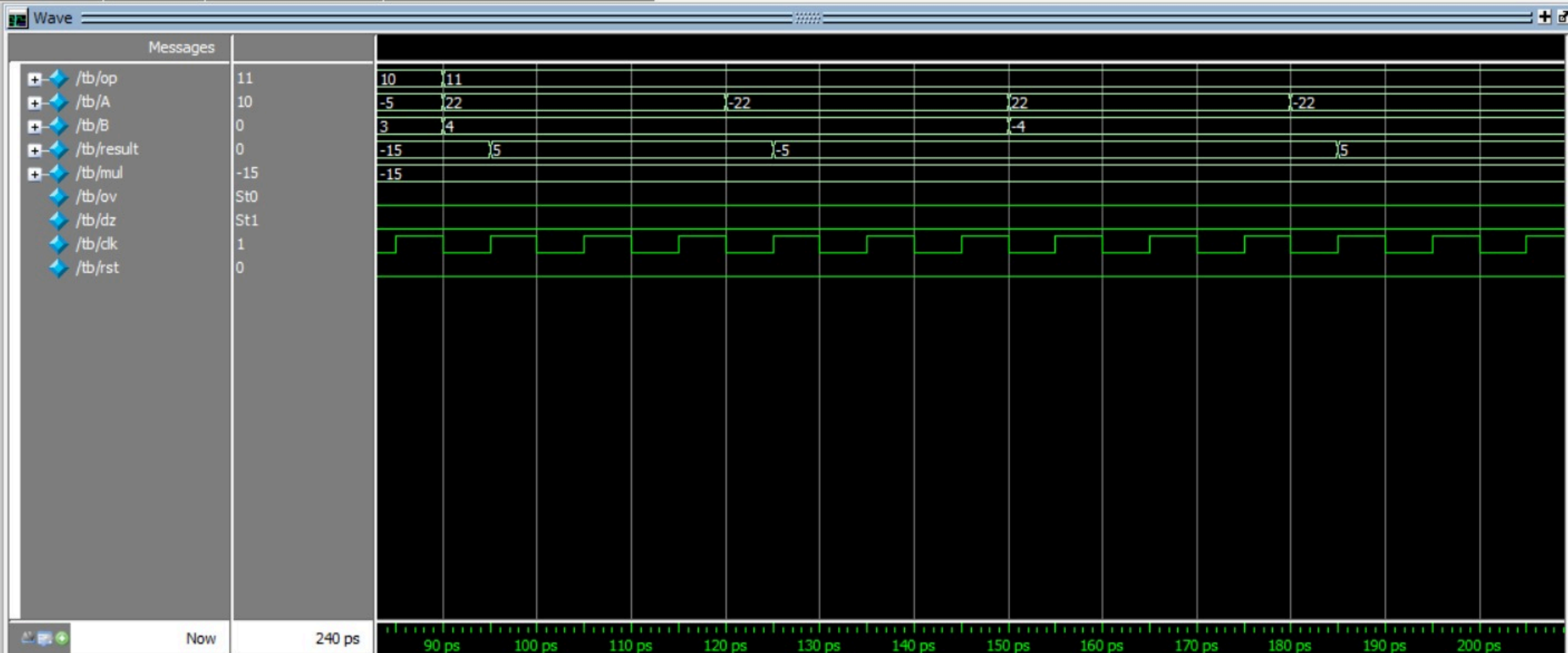
        // MUL signed
        op=2'b10; A=-5; B=3; #20;

        // DIV
        op=2'b11; A=22; B=4; #30;

        // DIV signed
        op=2'b11; A=-22; B=4; #30;

        $finish;
    end
endmodule

        $finish;
    end
endmodule
```

Messages													
+ /tb/op	11	00		01		10				11			
+ /tb/A	10	20		50		10		-5		22			-22
+ /tb/B	0	6		4		8		3		4			
+ /tb/result	0	0	26		46		80	-15		5			-5
+ /tb/mul	-15	0				80		-15					
/tb/ov	St0												
/tb/dz	St1												
/tb/clc	1												
/tb/rst	0												

Concluzie

În urma simulărilor realizate, ALU-ul proiectat a demonstrat funcționarea corectă pentru toate operațiile implementate: adunare, scădere, înmulțire și împărțire. Rezultatele obținute în testbench au fost conforme cu valorile teoretice așteptate, atât pentru cazuri simple, cât și pentru scenarii multiple de test. Multiplexorul controlat prin semnalul opcode a selectat corect rezultatul corespunzător fiecărei operații, confirmând integrarea corectă a modulelor.

O dificultate a fost gestionarea diferenței de dimensiune între rezultatele operațiilor. În timp ce adunarea și scăderea produc rezultate pe 8 biți, înmulțirea generează un rezultat pe 16 biți. Această problemă a fost rezolvată prin extinderea rezultatelor mai mici la 16 biți folosind concatenare, asigurând astfel consistența ieșirii ALU-ului.

De asemenea, a fost necesară tratarea cazului de împărțire la zero, care poate produce rezultate nedeterminate în simulare. Această situație a fost gestionată prin introducerea unei condiții de verificare, astfel încât rezultatul să fie setat la zero atunci când divizorul este zero.

În concluzie, problemele întâmpinate au fost de natură structurală și de compatibilitate a semnalelor, iar rezolvarea acestora a contribuit la obținerea unui design stabil și funcțional.



Thank You!